2024

Pràctica 1

Seguretat en els sistemes operatius

Sistemes Operatius II

Cristian Prado Méndez
Jan Accensi March

Índex

Índex **2**
Introducció 3
Preguntes i respostes 3
PRIMERA PART 3
SEGONA PART 4
Conclusions 7

Introducció

La seguretat en els sistemes operatius (SOs) és fonamental en l'era digital, especialment en SOs multiusuari com Linux. Aquests sistemes gestionen recursos i garanteixen un entorn segur per a l'execució d'aplicacions. Protegir la integritat de les dades, la privacitat dels usuaris i la disponibilitat de serveis és essencial. Una amenaça comuna és el **desbordament de pila (stack overflow)**, que permet als atacants executar codi maliciós i comprometre el sistema. Implementar mesures com la segmentació de privilegis, els pegats, el monitoratge i l'educació és vital per garantir la seguretat dels SOs.

Preguntes i respostes

PRIMERA PART

Pregunta 1: Mitjançant la línia de comandes, com podem mostrar els usuaris i grups del sistema?

```
awk -F: '{ print $1 }' /etc/passwd
```

Aquesta comanda ens retorna únicament una llista amb els usuaris. Si volem més informació (per exemple els grups) podem explorar tot l'arxiu /etc/passwd amb la línia de codi cat /etc/passwd. Els camps que es mostren en cada usuari signifiquen el següent: En el primer camp trobem el nom d'usuari, en el segon camp la contrasenya xifrada, que per motius de seguretat no es mostra i apareix una "x". En el tercer camp es mostra el User ID i en el quart el Group ID. Finalment, al cinquè camp es troba un comentari o informació respecte a l'usuari, al sisè el directori de treball i al setè la shell d'inici.

Pregunta 2: Què ha passat? Per a què serveix la comanda su? En quines situacions seria interessant fer-lo servir? Què passa si obro una nova terminal?

Un cop s'ha executat la comanda `su oslabadmin`, el terminal ha demanat la contrasenya d'aquest usuari i s'ha iniciat sessió. La comanda *su* serveix per canviar l'usuari en una sessió de terminal. Ens permet canviar tan l'UID com el GID.

És interessant fer servir aquesta comanda quan cal canviar d'usuari per executar determinades tasques amb privilegis específics. Quan s'inicia una nova sessió de terminal, es torna a canviar a l'usuari original, és a dir el que estem fent servir a la interfície gràfica. Això demostra que la comanda su només serveix per a canvis temporals i si realment necessitem treballar amb oslabadmin, és millor que canviem d'usuari el sistema.

Pregunta 3: Com podem veure les propietats del document creat (ejemplo.txt)? Qui és el propietari del fitxer? A quin grup pertany? Quins permisos tenen el propietari, el grup i altres usuaris sobre aquest fitxer?

Per a veure les propietats del document ejemplo.txt podem utilitzar el comandament ls. Personalment, nosaltres hem utilitzat ls -lisah ejemplo.txt perquè ens dona una sortida molt completa. Obtenim això

```
542173 4.0K -rw-rw-r-- 1 oslabadmin oslabadmin 25 Sep 19 17:56 ejemplo.txt
```

El propietari de l'arxiu es oslabadmin (1r oslabadmin indica això), el grup al que pertany és oslabadmin (2n oslabadmin indica això). Els permisos que té el propietari son rw- (llegir i escriure, no executar), els permisos que tenen els usuaris del grup son rw- (llegir i escriure, no executar). Els permisos que tenen els altres usuaris son r-- que vol dir que només són de lectura.

Pregunta 4: [Obrim i modifiquem arxiu ejemplo.txt des de la interfície] Què passa en intentar desar les modificacions? Per què?

Si s'intenta guardar les modificacions, el que un es troba és que no és possible, ja que s'ha iniciat la sessió amb l'usuari oslab i aquest no pertany al grup d'oslabadmin, que es diu oslabadmin. Per tant, només té permisos de lectura, i no pas d'escriptura.

Pregunta 5: [Es canvien els permisos d'ejemplo.txt] Pots veure el contingut del fitxer? Per què? Quina acció s'ha fet amb el chmod 700 ./ejemplo.txt?

No es pot veure el contingut d'aquest fitxer. Després de fer ús de la comanda `chmod 700 ./ejemplo.txt` el que hem fet és donar permisos de lectura, escriptura i execució a l'usuari oslabadmin per aquest fitxer, però hem tret tots 3 permisos a usuaris del grup i a altres usuaris fora del grup, en particular a l'usuari oslab. Per tant, a causa d'aquesta comanda el fitxer no es pot visualitzar.

Pregunta 6: Per a canviar permisos en directoris, quina comanda utilitzaries?

Primer de tot crearia el directori amb la comanda *mkdir SO2.* Seguidament faria ús de la comanda *chmod 770 SO2,* de forma que s'estan donant permisos de lectura, escriptura i execució per l'usuari oslabadmin i pel grup on pertany. D'aquesta forma, es restringeix l'accés a usuaris que no siguin oslabadmin o del seu grup, com oslab. Si volem, també podem canviar els permisos de tots els arxius de la carpeta de manera recursiva. Per a fer-ho utilitzem l'opció -R. És a dir, executem `chmod 770 -R SO2`

Pregunta 7: Com a darrer pas, crea un script a /tmp amb l'usuari oslabadmin (pots utilitzar algun dels implementats a SO1). Intenta executar-lo com a oslab. No hauries de poder en no disposar de permisos suficients. Com canviaràs aquests permisos per permetre l'execució de l'executable/script?

S'ha anomenat el script com holamundo.sh.
Primer de tot, des de l'usuari oslabadmin hauríem de fer ús de la comanda *chmod o+x holamundo.sh.* D'aquesta forma, el que estem fent és donar permisos d'execució a altres usuaris fora del grup com oslab. Seguidament, fem ús de la comanda *su oslab* per canviar d'usuari i executem el script fent ús de la comanda *./holamundo.sh.* Malgrat això, si provem d'executar l'arxiu només amb permisos d'execució, veurem que no podem. Això és perquè per tal de poder executar l'script també necessita permisos de lectura. (Si fos un arxiu binari, el podríem executar sense permisos de lectura, però com que el script.sh és un fitxer de text, també necessitem permisos de lectura).

SEGONA PART

Pregunta 8: A què es deu aquest missatge? Per què apareix amb "Sistemes Operatius" i no per a les altres contrasenyes? Per a la contrasenya de "Sistemes Operatius", com podries evitar aquest core dumped (sense modificar el codi .c)?

A l'hora de compilar apareix el warning de que la funció gets és vulnerable. Un cop entrada la contrasenya, apareix el missatge de stack smashing. Aquest missatge es deu perquè la variable buff té una mida de 15 bytes i cada lletra és d'1 byte. El que passa amb "SistemasOperativos" és que supera la mida que pot emmagatzemar la variable buff i això provoca que salti l'avís. Les contrasenyes "Chocolate" o "CafeConLeche" no provoquen aquest core dumped perquè no superen la mida del buffer. Per evitar el missatge tenim dues opcions. O bé augmentar la mida que pot emmagatzemar la variable buff (opció segura) o bé, si no volem modificar l'arxiu .c, compilar-lo desactivant el canary (opció molt perillosa). És a dir executant: `gcc stack4.c -o stack4 -fno-stack-protector`

Pregunta 9: Un cop deshabilitat el missatge de stack smashing detected, analitza el comportament del programa per a les diferents contrasenyes anteriors i justifica aquest comportament.

Un cop deshabilitat el canary, si provem executant el codi, observem que es mostra per pantalla el text "Privilegios de root asignados al usuario", indicant que hem sobreescrit en posicions de memòria invàlides. El que està passant és que com que la variable pass és contigua a la variable buff, quan aquesta s'omple, continua escribint a les posicions de memòria següents, és a dir a la variable pass. D'aquesta manera, la variable pass != 0 i entra dins del if per permisos root.

Pregunta 10: Fixem-nos primer a l'arxiu stack4.c. Compila el programa i analitza les possibles vulnerabilitats. Per explotar aquestes vulnerabilitats i desbordar el buffer, es proveeix un altre fitxer en python anomenat stack4_exploit.py. Analitza el resultat en fer la canonada python2 stack4_exploit.py | ./stack4. P10. Descriu què fa l'exploit?

Les possibles vulnerabilitats són a la funció gets, que com bé indica el compilador, no és segura. Quan mirem la documentació de la funció utilitzant: man gets, s'explica com funciona. La funció gets obé una cadena de caràcters de l'entrada estàndard. Cada caràcter s'escriu en un buffer fins que la funció detecta un EOF o un salt de línia. Finalment, quan es detecta un d'aquests caràcters, se substitueix per un \0 per acabar la cadena. La funció no controla quants caràcters escriurà al buffer i si nombre de caràcters d'entrada és superior a la mida del buffer, la funció gets continuarà escrivint a la següent ubicació de memòria.

El que està fent l'exploit és aprofitar-se d'aquesta vulnerabilitat. L'exploit utilitza el caràcter A, (x41 en hex), però podria utilitzar qualsevol caràcter. En primer lloc, s'omple el buffer (A*60) i s'afegeixen 3*8 cadenes addicionals de A. En el nostre cas, com que utilitzàvem un Mac amb xip ARM, hem hagut d'adaptar aquests caràcters addicionals a la nostra arquitectura. Per tal d'entendre per què hi ha aquestes 3 cadenes podem utilitzar l'eina gdb, que ens permet dissociar els mètodes compilats en codi assemblador. Seguint la documentació de l'article https://adamhlt.com/exploit-education-phoenix-stack-four/, hem vist que només necessitem 2*8 caràcters addicionals. El resultat de la canonada és que la return address passa a ser 0x11223344 i segmentation fault, perquè l'adreça és invàlida. L'exploit el que fa és canviar l'adreça de retorn del mètode start_level amb l'adreça arbitrària que li posem al exploit.

Pregunta 11: Fixa't que requereix python 2.7 (que hauràs d'instal·lar a la màquina virtual) i no Python 3 (instal·lat per defecte a Ubuntu). Per què no funciona amb python3 i si amb python2?

En primer lloc, es té que a python2, els strings són tractats com un array de bytes, que permet molta precisió a l'hora de configurar els exploits. D'aquesta forma aconsegueixen canviar l'adreça de retorn de la funció. En canvi, a python3, els strings són de tipus Unicode i l'exploit només funcionarà si es fan una sèrie de modificacions com codificar o descodificar, provocant algunes incompatibilitats amb

els exploits. Finalment, com a curiositat, tenim que la funció print en python2 no requereix parèntesis, però a python3 sí que en requereix.

Pregunta 12: Ara modifica el fitxer stack4_exploit.py perquè en executar la canonada anterior es truqui a la funció complete_level. Per això has d'identificar quina és la seva adreça en memòria (Pista: readelf -s) i, a continuació, sobreescriure-la a l'adreça de retorn a través del fitxer python. Aquesta direcció de la funció s'ha de proporcionar a Big Endian o Little Endian? Quina és la diferència entre elles? Com pots esbrinar si l'arquitectura de la màquina utilitza un o altre?

L'adreça de memòria de la funció complete_level s'haurà de proporcionar en Little Endian, ja que si fem ús de la comanda `lscpu | grep "Byte Order"` podrem veure que aquesta màquina treballa amb Little Endian. La diferència entre elles és que en el cas de Big Endian el byte més significatiu es col·loca primer, mentre que en Little Endian, el byte menys significatiu és el que es posa primer. Per tant, si suposem una adreça 0x11223344, en Big Endian seria \x11\x22\x33\x44, mentre que en Little Endian seria \x44\x33\x22\x11.

Pregunta 13: Recorda que hauràs de desactivar els diferents mecanismes existents per defecte que eviten el stack overflow. Descriviu el procés realitzat per explotar correctament el fitxer i que surti el missatge de Congratulations, you've finished phoenix/stack-four :-) Well done!

Primer de tot hem modificat l'arxiu stack4_exploit.py de forma que hem canviat l'adreça de retorn a la que ens ha proporcionat la comanda `readelf -s ./stack4 | grep complete_level`. Així hem obtingut l'adreça de la funció *complete_level* i l'hem canviat per la que hi havia a l'arxiu python en format Little-Endian. Seguidament hem desactivat tots els mecanismes que hi havia per evitar el stack overflow, fent ús de les comandes `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space` (per ara, realment no és necessàri) i `gcc -fno-stack-protector -z execstack -no-pie -o stack4 stack4.c`, fent ús dels flags per desactivar els mecanismes. L'arxiu python ens ha quedat de la següent forma:

```
LLETRA_A = "\x41"
exploit = LLETRA_A * 64  # buffer overflow
exploit += LLETRA_A * 8  # additional data
exploit += LLETRA_A * 8  # for the saved RBP
exploit += "\x84\x06\x40\x00\x00\x00\x00\x00"  # return adress
print exploit
```

Hem modificat l'adreça i també (com hem explicat a la pregunta 10) hem esborrat una línia de codi que sumava a l'exploit. D'aquesta forma hem aconseguit el següent output:

```
oslab@oslab:~/Documents/SO2/p1-codigo$ python2 stack4_exploit.py | ./stack4
Welcome to phoenix/stack-four, brought to you by https://exploit.education
and will be returning to 0x400724
Congratulations, you've finished phoenix/stack-four :-) Well done!
```

Pregunta 14: Executa el programa C sense introduir cap caràcter per teclat. A continuació observa el mapa de segments del procés en qüestió mitjançant l'ordre cat /proc//maps (en un altre terminal). PID correspon a l'identificador del procés que ben segur podreu esbrinar. Què podeu dir el rang d'adreces i permisos de la pila?

Observant aquesta línia: ffffccd4d000-ffffccd6e000 rw-p 00000000 00:00 0 [stack]

Podem veure que el rang d'adreces de la pila és desde ffffccd4d000 fins ffffccd6e000. En particular, tenim que la mida del rang és de ffffccd6e000 - ffffccd4d000 = 20000 hexadecimal = 131072 bytes

(128 KB). Respecte als permisos de la pila, podem concloure de rw-p que es tenen permisos de lectura i escriptura, però no d'execució. Si tornem a executar el programa, veiem que aquest rang canvia. Això és gràcies a la seguretat ASLR.

Pregunta 15: A continuació cal desactivar les mesures de seguretat ASLR i assignar permisos a la pila (veure setarch i execstack) perquè la variable buffer de stack5.c tingui sempre la mateixa adreça de memòria, i ens permeti escriure-hi. P15. Descriu el procés per desactivar aquestes mesures de seguretat.

Per tal de desactivar el ASLR, primer necessitem estar amb un usuari amb permisos com podria ser oslabadmin. I després hem d'executar echo 0 | sudo tee /proc/sys/kernel/randomize_va_space. També podríem desactivar ASLR només per l'executable stack5 amb la comanda setarch `uname -m` -R ./stack5

Per tal de poder executar a la pila, hem de compilar l'arxiu amb el flag -z execstack. Per exemple: `gcc -fno-stack-protector -z execstack -no-pie -o stack5 stack5.c` Podem comprovar que efectivament es pot executar en la pila amb la comanda `execstack -q stack5` i obtenim X stack5, que significa que podem executar a la pila.

Pregunta 16: Per acabar, executa la canonada python2 stack5_exploit.py | ./stack5 per conèixer la direcció estàtica de buffer i introdueix-la a l'stack5_exploit.py. Quin codi s'ha injectat? On és present aquesta injecció de codi a stack5_exploit.py?

```
oslab@oslab:~/Downloads/p1-codigo$ python2 stack5_exploit.py | ./stack5
Welcome to phoenix/stack-five, brought to you by https://exploit.education
Buffer address: 0x7fffffffdff0
and will be returning to 0x7fffffffdff0
ejemplo1    stack4    stack4_exploit.py  stack5.c
ejemplo1.c  stack4.c  stack5             stack5_exploit.py
```

El codi que s'ha injectat és la crida a sistema execve(/bin/ls). El codi està en format assemblador. Si utilitzem la pàgina https://www.codeconvert.ai/assembly-to-c-converter Podem veure que el programa convertit quadra amb el nostre output.

```
#include <unistd.h>
int main() {
   char *argv[] = {"/bin/ls", NULL};
   execve(argv[0], argv, NULL);
   return 0;
}
```

Conclusions

La seguretat en els SOs, especialment Linux, és clau per protegir dades i serveis davant d'amenaces com el **buffer overflow** o desbordament de memòria en general. L'adopció de mesures robustes, com el control de privilegis, és essencial per prevenir atacs i mantenir l'estabilitat en l'entorn digital actual. La nostra experiència en aquesta pràctica ha estat complicada, ja que en tenir tots dos membres de la parella un Macbook amb xip ARM, no en podíem fer ús de la comanda *execstack*, així com altres irregularitats comentades amb el professorat. Així i tot, hem pogut realitzar la pràctica fent ús d'un ordinador extern. Cal comentar que també hem adaptat l'exploit perquè funcioni en xip ARM.

La feina ha estat repartida al 50% entre els 2 companys.